Jonathan A.

Software & System Requirements

Table of Contents

Jonathan A.

What is the system, and what does it do?

The system that will be made is a pacemaker. Pacemakers were first developed in the mid 1900s. They are cardiovascular medical devices that treat abnormal heart rhythms, or so called “heart arrhythmia”. They’re widely used today, and can last between 8-10 years per device used on a patient. Pacemakers treat abnormal heart rhythms by sending an electrical pulse to the heart when the pacemaker detects any type of arrhythmia. It uses thin insulated wires that are connected to the heart. If the pacemaker detects that the heart rhythm is too slow, it will wait an interval of one second before sending a 2-5 volt pulse in which the patient cannot feel. It's almost as if their heart is beating normally!

Why am I making the system and what does it solve?

I am making the pacemaker because I’ve always been fascinated by its design, I’m also very interested in health and the medical field, as I research a ton about it, and I’m very into fitness. Another big reason is because I currently work with safety critical systems, and pacemakers fit perfectly in that category between my interest in the medical field, and my current career as an embedded software engineer for safety critical systems. Hazard analysis is extremely important when dealing with systems that can potentially harm an individual when any type of fault occurs. I think as engineers we need to make sure that we mitigate these risks and understand the potential hazards that can come with it when a fault happens. So understanding the pacemakers risks, how they are developed, and how to mitigate those risks is why I’m developing a pacemaker as a personal project.

Hardware Parts needed for the system(roughly)

- **Potentiometer:** this will act in place of a humans heart, turning the knob clockwise will increase the heart BPM by increasing voltage. Counter clockwise will do the opposite.
- **STM32 with ARM Cortex-M4 Core:** This is the devboard we will be using to receive potentiometer data via the Analog-to-Digital converter line, calculate that data into heartbeats, determine whether the heartbeats are low enough to send an electrical pulse, then send an electrical pulse to an LED. it will also make an ECG and send diagnostics data to an OLED display via I2C.

- **LED:** a simple LED to indicate when an electrical pulse was sent by the pacemaker.
- **OLED Display**: a small display that will receive and displays an ECG(Electrocardiogram), and general diagnostics, like heartrate.(May add more)

Simulating battery life

Whether the STM32 is running on battery or not, we want to simulate a real world pacemaker as much as we can. That's why not only are we doing hazard analysis, state machines, etc. but we will also be ensuring that we are not burning the pacemaker's battery life when we don't need to.***(As real world pacemakers live 8-10 years on average!)*** Making the CPU go to sleep when it's not being used, aka, when the heart rate we're receiving is beating at a normal, healthy rate. Our Processor should only be awake when the heart rate is too slow, and abnormal.

Why a Potentiometer?

A potentiometer is a very good choice to simulate a heart rate because it is a controllable, **testable** and simple way to represent a biological signal, without the need of an actual heart. We want to be able to test and prove both the firmware logic, and our hardware as much as we can.

General Architecture

In summary it will look something like this, Analog voltage → ADC reading → Software decision, whereas a real pacemaker would look something like this, Patient signal/heart rate → Sensor → Analog front-end → ADC Reading → Software decision. These are extreme simplified versions of it, so its best not to think of this as the entirety of the architecture.